Author Instructions for IMVIP 2019

Michael McDonagh

Anonymous Affiliation

Abstract

This study explores the development of a deep learning system to classify 19 different types of food. Recognising food from images is a difficult task because many dishes look very similar and the way a meal is plated can vary significantly. This research compares two main methods: a custom-built Convolutional Neural Network (CNN) and a Transfer Learning approach that uses a pre-trained model as its foundation.

Keywords: Imaging, Image Processing, Machine Vision, etc. (Maximum five)

1 Introduction

Initial attempts with a custom model resulted in clear underfitting, where the accuracy was no better than random guessing at 5.2%. By improving the model structure, specifically by using double convolutional blocks, increasing the number of filters to 128, adding Dropout Layers to prevent over-reliance on specific features, the custom model eventually achieved a stable validation accuracy of 65%. To try and improve further, a two-phase transfer learning strategy was used. This involved an initial phase where the base model was frozen, followed by a fine-tuning phase to adapt the weights to this specific food dataset.

Figure 1: Introduction Banner.

2 Data and Pre-Processing

The first thing I did was to initialise any of the variables I will be using that will detail how I will be processing my data. The seed that I set is `421944` for my student number, this will allow for reproducability of my results. I then have the batch sizes for my data in batches of `32`, dealing with image sizes of `128x128` and `64x64`. Computational tasks were performed locally using an NVIDIA GeForce RTX 4060 GPU, with a verification step included in the script to confirm CUDA acceleration was active.

2.1 Dataset Description

The image dataset being used is a subset of the Food 101 dataset containing 19 folders named after the type of food contained in its images. The images in use are `512x512`, but will be rescaled across `128x128` and `64x64`, grayscaled and/or augmented for the purpose of exploring different hyperparameters and comparing models.

2.1.1 Preprocessing and Splits

The dataset contained images with 3 colour channels (RGB), since that would involve numbers ranging from 0.0 to 255, it was important the training data was normalised. Normalisation will help with data moving between layers, and help prevent vanishing/exploding gradients. The training split that was chosen was to split the Training Set at 30% (13,312 images), Validation 10% (1,888 images), for tuning the hyperparameters and the final Test Set at 20% (3,840 images). The final Test Set is reserved to the final evaluation of the model.

- Training Set at 30% (13,312 images) - Used for model weight updates
- Validation 10% (1,888 images) - Hyperparameter Tuning
- Test Set at 20% (3,840 images) - Reserved to the final evaluation of the model.

2.1.2 Exploratory Analysis

An initial audit of the dataset confirmed it is perfectly balanced. Each of the 19 classes contains exactly $1,000$ images, totaling $19,000$ samples with 3 channels and 128x128 in size. This eliminates the risk of class imbalance, where a model might become biased toward a majority class to artificially inflate accuracy. In such a case where there was an imbalance, class weights can be introduced to give equal importance to all classes.

2.2 Experiments

Sed ut perspiciatis, unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam eaque ipsa, quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt, explicabo. Nemo enim ipsam voluptatem, quia voluptas sit, aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos, qui ratione voluptatem sequi nesciunt, neque porro quisquam est, qui dolorem ipsum, quia dolor sit amet consectetur adipisci[ng] velit, sed quia non numquam [do] eius modi tempora inci[di]dunt, ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur? Quis autem vel eum iure reprehenderit, qui in ea voluptate velit esse, quam nihil molestiae consequatur, vel illum, qui dolorem eum fugiat, quo voluptas nulla pariatur?

2.3 CNN From Scratch

The first CNN model, **Model V1** I had developed was configured with two convolutional blocks for feature extract and a single MaxPooling2D for downsampling after each. With the initial baseline model I had observed that it had suffered from extreme overfitting, having a training accuracy of 99% while the validation accuracy was stuck around 20%. This clear gap showcases that the model was memorising the training data and could not generalise on new unseen data.

The next iteration with **Model V2** focused on combatting the overfitting problem by introducing a total of three convolutional blocks, doubling each layer to 2 instead of 1 so it can focuse on fidning more features, dense increased to 256, and also introducing Dropout at '0.5' as this would force the network to not rely on any specific node. The model resulted in val accuracy at 0.055% followed by train accuracy at 0.053%, showcasing the model moving in the opposite direction, from overfitting, to underfitting.

Finding the perfect balance where the model can learn patterns and generalise on new unseen data is key, so in **Model V3**, I had scaed back to 2 convolutional blocks with each containing batch normalisation to prevent numbers becoming too large as they move through the layers, and then the MaxPooling2D. GlobalAveragePooling2D was also used to reduce the number of parameters down from '8,680,755' to '41,299'. This model iteration showed the most progress so far, where the training accuracy is smoothly increasing and loss is smoothly decreasing. Furthermore, the validation loss is decreasing with some stagnation, and accuracy is increasing with that same stagnation but keeping close in line with the respective train lines.

Moving onto **Model V4**, we introduce image augmentation, to modify the dataset images to help improve generalisation. The augmentation added were to flip, rotate and zoom the dataset images. This improved the stability of the training loss but introduced significant volatility in the validation metrics. The final validaiton accuracy of 38% suggests that while the model is learning general features, it remains sensitive to extreme image transformations.

The models **V5** and **V6** respectively, focused on inreasing the convolutional layer filters to help detect more complex semantic features, with **V5** having a max '256' filter and **V6** '1024' filter, further pushing the dense layer towards '2048'. **V6** had been able to increase the validation accurcy to 67% as a result.

In **Model V7**, the architecture was further deepened to include a 512-filter stage while significantly increasing the Dropout rate to 0.6 1 and reducing the learning rate to 0.0001 1. The objective was to prioritise stability and prevent the volatility observed in earlier versions. While the training curves appeared smoother (as seen in Figure 2), the model achieved a final validation accuracy of approximately 60%, failing to surpass the performance of the shallower V6 architecture1.

The final iteration, **Model V8**, took inspiration from the high-capacity filter banks of V6 but extended the training duration to 100 epochs and expanded the dense layer to 2048 units. Despite the additional training time and increased model capacity, the validation accuracy plateaued early and eventually began to diverge, as seen in the widening gap between the training and validation loss curves in Figure 2.

Ultimately, **Model V6** was identified as the superior architecture developed from scratch. It provided the optimal balance of depth and width, achieving a peak validation accuracy of $\approx 67\%$ 1 without the severe overfitting seen in V8 or the over-regularisation of V7. This model served as the final benchmark for our custom-built architectures before transitioning to transfer learning techniques.

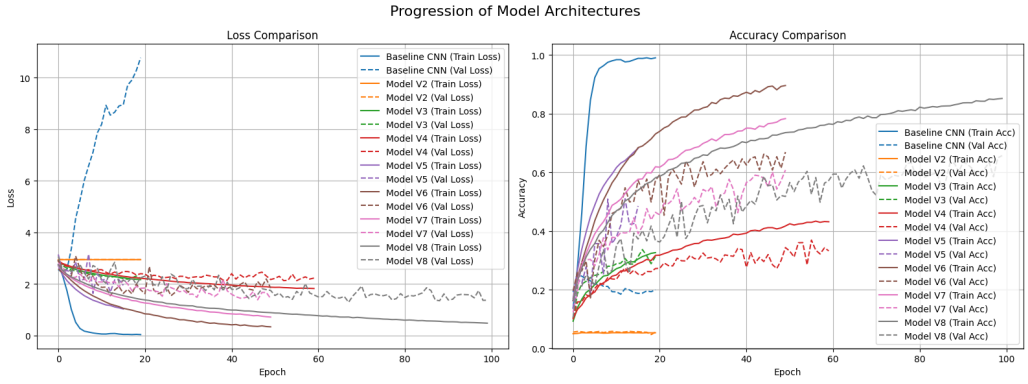

Figure 2: Performance Comparison of CNN Architectures from Scratch

Table 1: Model Performance Comparison Tracker. Improvement metric is based on comparison with the Baseline CNN Model

Model Version	**Time (s)**	**Final Train Acc**	**Final Val Acc**	**Final Val Loss**	**Improvement (%)**
Baseline CNN	1029.10	0.9899	0.1992	10.7718	0.00
Model V2	1029.10	0.0529	0.0551	2.9453	-14.41
Model V3	1029.10	0.3261	0.3130	2.2151	11.38
Model V4	1029.10	0.4312	0.3321	2.2290	13.29
Model V5	1029.10	0.6811	0.4852	1.9460	28.60
Model V6	1029.10	0.8955	0.6690	1.8402	46.98
Model V7	2033.53	0.8514	0.6568	1.3706	45.76
Model V7	2033.53	0.8514	0.6568	1.3706	45.76

2.4 Transfer Learning

Sed ut perspiciatis, unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam eaque ipsa, quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt, explicabo. Nemo enim ipsam voluptatem, quia voluptas sit, aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos, qui ratione voluptatem sequi nesciunt, neque porro quisquam est, qui dolorem ipsum, quia dolor sit amet consectetur adipisci[ng] velit, sed quia non numquam [do] eius modi tempora inci[di]dunt, ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur? Quis autem vel eum iure reprehenderit, qui in ea voluptate velit esse, quam nihil molestiae consequatur, vel illum, qui dolorem eum fugiat, quo voluptas nulla pariatur?

2.5 Results and Comparison

Sed ut perspiciatis, unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam eaque ipsa, quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt, explicabo. Nemo enim ipsam voluptatem, quia voluptas sit, aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos, qui ratione voluptatem sequi nesciunt, neque porro quisquam est, qui dolorem ipsum, quia dolor sit amet consectetur adipisci[ng] velit, sed quia non numquam [do] eius modi tempora inci[di]dunt, ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur? Quis autem vel eum iure reprehenderit, qui in ea voluptate velit esse, quam nihil molestiae consequatur, vel illum, qui dolorem eum fugiat, quo voluptas nulla pariatur?

2.6 Real-World Test

Sed ut perspiciatis, unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam eaque ipsa, quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt, explicabo. Nemo enim ipsam voluptatem, quia voluptas sit, aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos, qui ratione voluptatem sequi nesciunt, neque porro quisquam est, qui dolorem ipsum, quia dolor sit amet consectetur adipisci[ng] velit, sed quia non numquam [do] eius modi tempora inci[di]dunt, ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur? Quis autem vel eum iure reprehenderit, qui in ea voluptate velit esse, quam nihil molestiae consequatur, vel illum, qui dolorem eum fugiat, quo voluptas nulla pariatur?

2.7 Conclusion

Sed ut perspiciatis, unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam eaque ipsa, quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt, explicabo. Nemo enim ipsam voluptatem, quia voluptas sit, aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos, qui ratione voluptatem sequi nesciunt, neque porro quisquam est, qui dolorem ipsum, quia dolor sit amet consectetur adipisci[ng] velit, sed quia non numquam [do] eius modi tempora inci[di]dunt, ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur? Quis autem vel eum iure reprehenderit, qui in ea voluptate velit esse, quam nihil molestiae consequatur, vel illum, qui dolorem eum fugiat, quo voluptas nulla pariatur?

2.8 Bibliographic references

References in a bib file format (e.g. imvip2019.bib given in the template) can be inserted using bibtex along with LaTeX/pdflatex (e.g. [**?**] or [**?**, **?**]).